

Aurum — security and performance audit, September 2026

Scope: every API route, every server-rendered page, the RLS policies, and the outbound notification path. Findings are ordered by severity. Everything marked **Fixed** is in this change; everything marked **Open** needs a decision from you.

1. Security

Fixed

#	Severity	Finding
S1	High	<code>POST /api/payments</code> checked that the caller was approved but never that the booking was theirs. Because the booking is read with the service-role client, RLS did not cover the gap: any approved employee could post an arbitrary booking UUID and attach a payment to a colleague's booking. Now gated on <code>created_by</code> , with Admin/Director exempt, and a Director alert on the deny path.
S2	High	<code>POST /api/users/approve/notify</code> had no authentication at all and took the target <code>userId</code> from the request body. Anyone on the internet could push notifications into every Director's inbox by posting UUIDs at it. It now derives the subject from the caller's own session, ignores the body, and only fires for an account that is genuinely <code>PENDING_APPROVAL</code> .
S3	Medium	<code>POST /api/bookings/[id]/customers</code> (Attach person) let any SM or CP attach a person to <i>any</i> booking — it read the booking with the RLS-scoped client, so it 404'd on a colleague's booking by luck rather than by design. Now reads with the service-role client and makes the ownership decision explicitly.
S4	Medium	Cross-employee access attempts were invisible. There is now an alerter (<code>lib/security/access-alert.ts</code>) wired into every deny path.

The access alerter

Wired into: the booking detail page, the payment detail page, the document download route, `POST /api/payments`, and the Attach person route.

On a blocked attempt it does three things:

1. Writes an `audit_logs` row with `action = 'ACCESS_DENIED'` — the permanent record, and the one thing that must not be lost.
2. Sends an in-app notification to **every approved Director**, priority `URGENT`. Never throttled.

3. Emails every Director, **throttled to one message per employee per 15 minutes**. The throttle is implemented as a time bucket inside the delivery row's dedupe key, so it costs no extra query — the existing unique index does the work. Without it, someone scripting UUIDs would generate thousands of emails, bury the signal, and exhaust the daily SMTP quota.

It swallows every error internally, so an alerting failure can never change the HTTP status the caller is about to return.

One design note worth knowing. The detail pages read through the RLS-scoped client, so a colleague's booking comes back as `null` — indistinguishable at the call site from a booking that was deleted or a mistyped URL. Alerting on every miss would generate constant false positives. So on the miss path only, `reportMissedRecordAccess()` re-reads with the service-role client: if the row genuinely exists and belongs to someone else, that is an unambiguous cross-employee attempt and the Directors are told; if it does not exist, nothing happens. Costs one extra query, and only when the read already failed, so the normal case is untouched.

Reviewer roles (Accountant, Admin, Director) are excluded — they can already see everything, so a miss for them is a real miss, not a boundary being tested.

Open — needs your decision

01. `/b/<uuid>` is public by design and exposes more than it needs to.

Customers have no accounts, so the booking page has no auth — that is deliberate and I have not changed it. But anyone holding the link can see the bank name, IFSC, branch, the full payment history, the audit trail, and the document list. The account number is masked; nothing else is. Options, cheapest first: (a) drop the bank block and the audit trail from that page, (b) add a short-lived signed token to the link, (c) leave it and accept that the URL is the secret. I would do (a) now regardless of which you pick.

02. No rate limiting anywhere. Login, registration, booking creation, payment submission and document download are all unthrottled. The access alerter reduces the blast radius of enumeration but does not stop it. This needs an edge rate limiter before you are handling real customer volume.

03. Storage bucket policies not reviewed. The document route is now correctly authorised, but if the `documents` bucket itself is readable with the anon key, that route can be bypassed entirely. Worth checking in the Supabase dashboard — I could not verify it from the codebase.

2. Performance

Fixed

#	Impact	Finding
P1	High	<code>/history</code> selected entire tables — every booking, every payment, every notification — with no <code>order</code> and no <code>limit</code> , then merged them in JS. For a Director that is the whole database on every page load. It was also silently <i>wrong</i> : PostgreSQL applies its own max-rows ceiling, so an unordered uncapped query returns an arbitrary slice, not the newest one, and timeline entries went missing as the tables grew. Now ordered newest-first and capped at 200 per source.
P2	Medium	<code>/users/[id]</code> and <code>/u/[id]</code> ran eight uncapped queries each — every booking a person ever created, every payment across those bookings, every attachment, every ticket and comment. All capped now (100 bookings / 300 payments / 200 attachments / 300 comments).
P3	Medium	Approving a payment ran four Supabase round-trips in series — notification, audit write, superseded-email cancel, contact lookup — before the button came back. Nothing depended on anything else, so they now run together and the slowest one sets the latency instead of the sum.
P4	Low	<code>/tickets/[id]</code> read the whole comment thread uncapped. Capped at 500.
P5	Medium	Five missing indexes, added in migration <code>0019</code> . See below.

Indexes added (`0019_overdue_and_security_indexes.sql`)

- `idx_bookings_outstanding` — partial index on `(status, created_at) where remaining_balance > 0`. The daily overdue sweep would otherwise filter across every submitted booking and then sort.
- `idx_payments_approved_by_booking` — partial, `(booking_id, payment_date desc) where status = 'APPROVED'`. Answers "when did money last arrive on this booking" without a sequential scan per run.
- `idx_audit_action_created` — `(action, created_at desc)`. Makes "every ACCESS_DENIED, newest first" cheap for the Director review.
- `idx_booking_customers_booking` — the junction had an index on `customer_id` but every lookup goes the other way, and the primary-first ordering was an unindexed sort. This runs on every payment email now, so it matters.
- `idx_users_role_status` — the alerter reads every approved Director on each attempt.

Verified healthy — no change needed

- `/bookings` and `/payments` already paginate with `.range()`.
- `/verification` caps at 100 per queue.
- `/analytics` caps at 2000 bookings / 4000 payments.
- `/dashboard` uses the `get_dashboard_stats` RPC with a JS fallback, and runs its four queries in parallel.
- `getSessionUser()` is wrapped in React `cache()`, so the layout and the page share one auth round-trip.

- The proxy has a fast path that skips `auth.getUser()` entirely when there is no auth cookie.
- No N+1 loops in any page. The one `await` inside a loop (`app/api/bookings/route.ts`, inserting co-buyers) is bounded by the number of people on the form and is a write, so batching it would not help much.

Open

O4. There is no runtime performance telemetry. Everything above is static analysis — real p95 latency per route, database query duration, and Storage upload time are not measured anywhere. Vercel Analytics plus Supabase's query performance view would cover it, and until they are on, "is it slow?" can only be answered by clicking around.

3. Notes on a UI regression I did not touch

`app/globals.css` ends with a block that sets `scrollbar-width: none` and `::-webkit-scrollbar { display: none }` on `*`. It comes after — and therefore overrides — the visible-scrollbar rules earlier in the same file, so every scrollbar in the app is currently hidden again. That undoes the change you asked for previously. I have left it alone because it arrived with your own UI update and reversing it is your call, but if it was not deliberate, deleting the last block of `globals.css` restores the visible scrollbars.

4. Operational follow-ups

1. Run migration `0019` (safe to re-run, no enum changes, does not need to run alone).
2. Set `CRON_SECRET` in the Vercel project's environment variables. Generate it with `openssl rand -hex 32`. Vercel Cron sends it automatically as `Authorization: Bearer $CRON_SECRET`; without it set, only a signed-in Admin or Director can trigger the sweep by hand.
3. `vercel.json` schedules the overdue sweep at 03:30 UTC daily. On the Hobby tier Vercel allows one cron run per day, which is exactly what this job needs — no upgrade required.
4. No Edge Function redeploy is needed. The worker delivers pre-rendered payloads and knows nothing about templates, so all the email changes ship with the web app.